

Embedded Software

Nathan Warner



**Northern Illinois
University**

Computer Science
Northern Illinois University
United States

Contents

1	Introduction	2
2	Creating a system architecture	6
3	Getting your hands on the hardware	14
4	Inputs, outputs, and timers	21

Introduction

- **Embedded systems:** An embedded system is a computerized system that is purpose-built for its application.

Because its mission is narrower than a general-purpose computer, an embedded system has less support for things that are unrelated to accomplishing the job at hand. The hardware often has constraints. For instance, consider a CPU that runs more slowly to save battery power, a system that uses less memory so it can be manufactured more cheaply, and processors that come only in certain speeds or support a subset of peripherals.

The hardware isn't the only part of the system with constraints. In some systems, the software must act deterministically (exactly the same each time) or in real time (always reacting to an event fast enough). Some systems require that the software be fault-tolerant with graceful degradation in the face of errors. For example, consider a system in which servicing faulty software or broken hardware may be infeasible (such as a satellite or a tracking tag on a whale). Other systems require that the software cease operation at the first sign of trouble, often providing clear error messages (for example, a heart monitor should not fail quietly).

- **Compilers and Languages:** Embedded systems use cross-compilers. Although a cross-compiler runs on your desktop or laptop computer, it creates code that does not. The crosscompiled image runs on your target embedded system. Because the code needs to run on your processor, the vendor for the target system usually sells a cross-compiler or provides a list of available cross-compilers to choose from. Many larger processors use the cross-compilers from the GNU family of tools.

Cross-compiling is the process of building executable software on one system (the host) that is designed to run on a different system (the target). The opposite of cross-compiled is **natively compiled**.

Embedded software compilers often support only C, or C and C++. In addition, some embedded C++ compilers implement only a subset of the language (multiple inheritance, exceptions, and templates are commonly missing). There is a growing popularity for other languages but C and C++ remain the most prevalent.

Regardless of the language you need to use in your software, you can practice object-oriented design. The design principles of encapsulation, modularity, and data abstraction can be applied to any application in nearly any language. The goal is to make the design robust, maintainable, and flexible. We should use all the help we can get from the object-oriented camp.

- **Debugging:** If you were to debug software running on a computer, you could compile and debug on that computer. The system would have enough resources to run the program and support debugging it at the same time. In fact, the hardware wouldn't know you were debugging an application, as it is all done in software.

Embedded systems aren't like that. In addition to a cross-compiler, you'll need a cross-debugger. The debugger sits on your computer and communicates with the target processor through the special processor interface

The interface is dedicated to letting someone else eavesdrop on the processor as it works. This interface is often called **JTAG** (pronounced “jay-tag”), regardless of whether it actually implements that widespread standard.

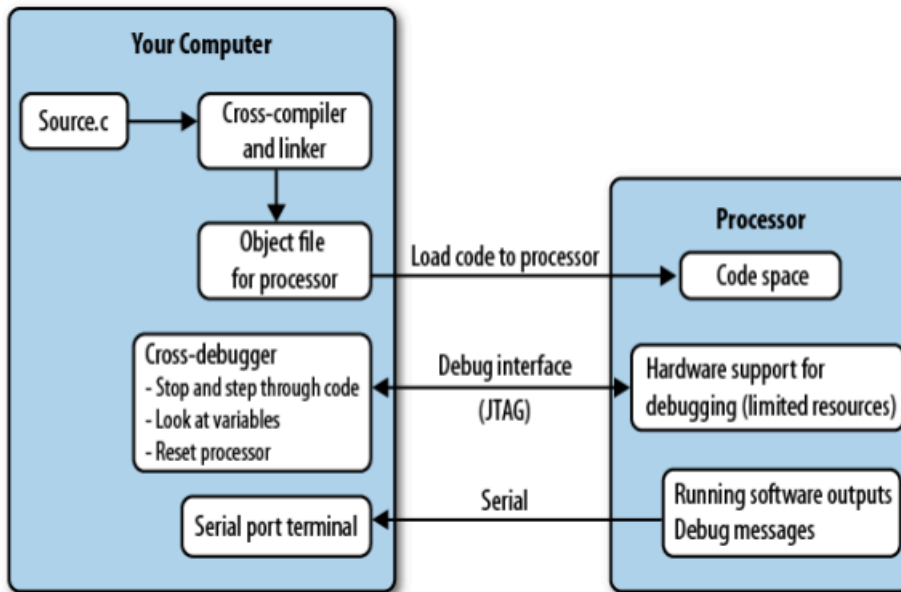


Figure 1-1. Computer and target processor

The processor must expend some of its resources to support the debug interface, allowing the debugger to halt it as it runs and providing the normal sorts of debug information. Supporting debugging operations adds cost to the processor. To keep costs down, some processors support a limited subset of features. For example, adding a breakpoint causes the processor to modify the memory-loaded code to say “stop here.” However, if your code is executing out of flash (or any other sort of read-only memory), instead of modifying the code, the processor has to set an internal register (hardware breakpoint) and compare it at each execution cycle to the address being run, stopping when they match. This can change the timing of the code, leading to annoying bugs that occur only when you are (or maybe aren’t) debugging. Internal registers take up resources, too, so often there are only a limited number of hardware breakpoints available (frequently there are only two).

Note: JTAG stands for **Joint Test Action Group**, which is the industry standard interface used for testing, programming, and debugging electronic circuit boards and microchips.

The device that communicates between your PC and the processor is generally called a hardware debugger, programmer, debug probe, in-circuit emulator (ICE), or JTAG adapter. These may refer (somewhat incorrectly) to the same thing, or they may be multiple devices. The debugger is specific to the processor (or processor family), so you can’t take the debugger you got for one project and assume it will work on another. The debugger costs add up, particularly if you collect enough of them or if you have a large team working on your system.

To avoid buying a debugger or dealing with the processor limitations, many embedded systems are designed to have their debugging done primarily via `printf` or some sort of lighter-weight logging to an otherwise unused communication port. Although incredibly useful, this can also change the timing of the system, possibly leaving some bugs to be revealed only after debugging output is turned off.

Writing software for an embedded system can be tricky, as you have to balance the needs of the system and the constraints of the hardware.

- **Resource constraints:** An embedded system is designed to perform a specific task, cutting out the resources it doesn't need to accomplish its mission. The resources under consideration include:
 - Memory (RAM)
 - Code space (ROM or flash)
 - Processor cycles or speed
 - Power consumption (which translates into battery life)
 - Processor peripherals

To some extent, these are exchangeable. For example, you can trade code space for processor cycles, writing parts of your code to take up more space but run more quickly. Or you might reduce the processor speed in order to decrease power consumption. If you don't have a particular peripheral interface, you might be able to create it in software with I/O lines and processor cycles. However, even with trading off, you have only a limited supply of each resource. The challenge of resource constraints is one of the most pressing for embedded systems.

Another set of challenges comes from working with the hardware. The added burden of cross-debugging can be frustrating. During board bring-up, the uncertainty of whether a bug is in the hardware or software can make issues difficult to solve. Unlike your computer, the software you write may be able to do actual damage to the hardware. Most of all, you have to know about the hardware and what it is capable of. That knowledge might not be applicable to the next system you work on. You will need to learn quickly.

Once development and testing are finished, the system is manufactured, which is something most pure software engineers never need to consider. However, creating a system that can be manufactured for a reasonable cost is a goal that both embedded software engineers and hardware engineers have to keep in mind. Supporting manufacturing is one way you can make sure that the system that you created gets reproduced with high fidelity.

After manufacture, the units go into the field. With consumer products, that means they go into millions of homes where any bugs you created are enjoyed by many. With medical, aviation, or other critical products, your bugs may be catastrophic (which is why you get to do so much paperwork). With scientific or monitoring equipment, the field could be a place where the unit cannot ever be retrieved (or retrieved only at great risk and expense; consider the devices in volcano calderas), so it had better work. The life your system is going to lead after it leaves you is something you must consider as you design the software.

- **Prototypes and Maker Boards:** The prototype is not the product. There are many tasks often forgotten at this stage. How will the firmware update? Does the system need to sleep to lower power consumption? Do we need a watchdog in case of catastrophic error? How much space and processing cycles do we need to save for future bug fixes and improvements? How do we manufacture many devices instead of one handbuilt unit? How will we test for safety? Inexplicable user commands? Corner cases? Broken hardware?

Software aside, when the existing off-the-shelf boards don't meet your needs, custom boards are often necessary. The development boards may be too delicate, connected by fragile wires. They may be too expensive for your target market or take too much power. They may be the wrong size or shape. They may not hold up under the intended environmental conditions (like temperature fluctuations in a car, getting wet, or going to space).

Whatever the reason for getting a custom board, it usually means removing the programming (and debugging) hardware that are part of processor development boards.

Custom hardware will also push you out of some of the simplified development environments and software framework. Using the microprocessor vendor's hardware abstraction layers with a traditional compiler, you can get to smaller code sizes (often faster as well). The lack of anything between you and the processor allows you to create deterministic and real-time handling.

Still, there is a chasm between prototype and shipping device, between supporting one unit on your desk and a thousand or a million in the field. Don't be lulled into believing the project is complete because all of the features finally worked (one time, in perfect conditions).

Development boards and simplified development environments are great for prototypes and to help select a processor. But there will be a time when the device needs to be smaller, faster, or cheaper.

Creating a system architecture

- **Getting Started:** There are two ways to go about architecting systems. One is to start with a blank slate, with an idea or end goal but nothing in place. This is composition. Creating these system diagrams from scratch can be daunting. The blank page can seem vast.

The other way is to go from an existing product to the architecture, taking what someone else put together to understand the internals. Reverse engineering like this is decomposition, breaking things into smaller parts

When you join a team, sometimes you start with a close deadline, a bunch of code, and very little documentation with no time to write more.

Diagrams will help you understand the system. Your ability to see the overall structure will help you write your piece of the system in a way that keeps you on track to provide good-quality code and meet your deadline.

You may need to consider formulating two architectures: one local to the code you are working on and a more global one that describes the whole product so you can see how your piece fits in.

As your understanding deepens, your diagrams will get larger, particularly for a mature design. If the drawing is too complex, bundle up some boxes into one box and then expand on them in another drawing.

Some systems will make more sense with one drawing or another; it depends on how the original architects thought about what they were doing.

- **Creating System Diagrams:** Just as hardware designers create schematics, you should make a series of diagrams that show the relationships between the various parts of the software. Such diagrams will give you a view of the whole system, help you identify dependencies, and provide insight into the design of new features

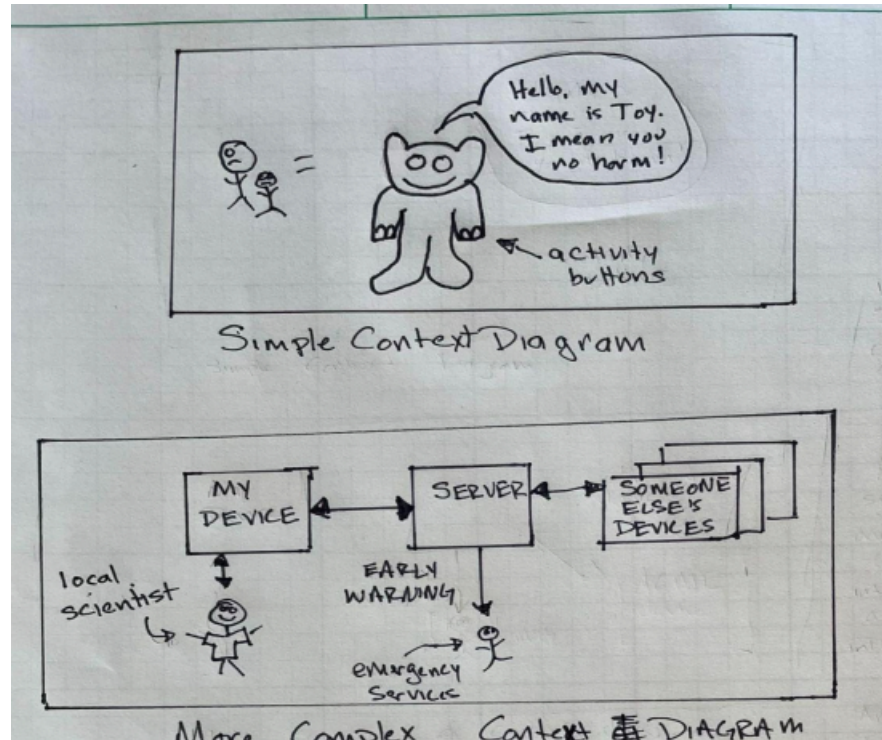
I recommend four types of diagrams:

1. Context diagram
 2. Block diagram
 3. Organigram
 4. Layering diagram
- **The Context Diagram:** The first drawing is an overview of how your system fits into the world at large. If it is a children's toy, the diagram may be easy: a stick figure and some buttons on the toy. If it is a device providing information to a network of seismic sensors with an additional output to local scientists, the system is more complicated, going beyond the piece you are working on.

The goal here is a high-level context of how the system will be used by the customer. Don't worry about what is inside the boxes (including the one you are designing). The diagram should focus on the relationships between your device, users, servers, other devices, and other entities.

What problem is it solving? What are the inputs and outputs to your system? What are the inputs and outputs to the users? What are the inputs and outputs to the system-at-large? Focus on the use cases and world-facing interactions.

Ideally this type of diagram will help define the system requirements and envision likely changes. More realistically, it is a good way to remember the goals of the device as you get deeper into designing the software.



- **The block diagram:** Design is straightforward at the start because you are considering the physical elements of the system, and you can think in an object-oriented fashion—no matter whether you are using an object-oriented language—to model your software around the physical elements. Each chip attached to the processor is an object. You can think of the wires that connect the chip to the processor (the communication methods) as another set of objects.

Start your design by drawing these as boxes where the processor is in the center, the communication objects are in the processor, and each external component is attached to a communication object.

For this example, I'm going to introduce a type of memory called flash memory. While the details aren't important here, it is a relatively inexpensive type of memory used in many devices. Many flash memory chips communicate over the SPI bus, a type of serial communication. Most processors cannot execute code over SPI, so the flash is used for off-processor storage. Our schematic, shown at the top of Figure 2-2, shows that we have some flash memory attached to our processor via SPI.

Ideally, the hardware block diagram is provided by an electrical engineer along with the schematics to give you a simplified view of the hardware. If not, you may need to sketch your own on the way to a software block diagram.

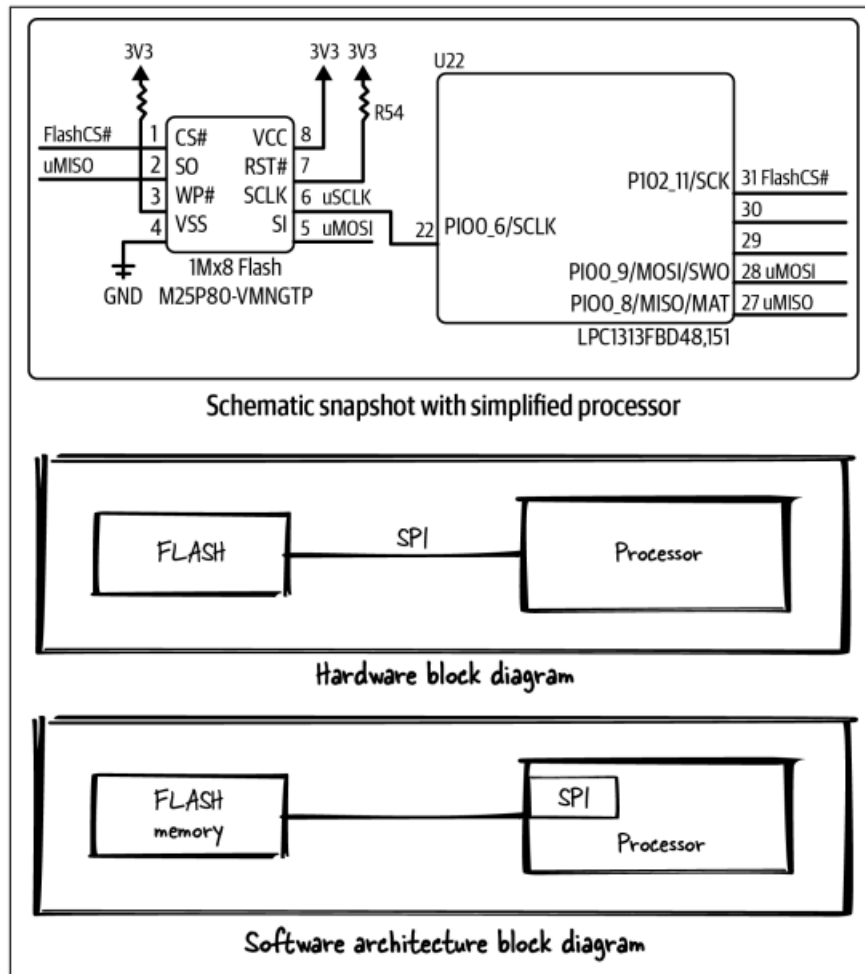


Figure 2-2. Comparison of schematic and initial hardware and software block diagrams

In our software block diagram, we'll add the flash as a device (a box outside the processor) and a SPI box inside the processor to show that we'll need to write some SPI code. Our software diagram looks very similar to the hardware schematic at this stage, but as we identify additional software components, it will diverge.

The next step is to add a flash box inside the processor to indicate that we'll need to write some flash-specific code. It's valuable to separate the communication method from the external component; if there are multiple chips connected via the same method, they should all go to the same communications block in the chip. The diagram will at that point warn us to be extra careful about sharing that resource, and to consider the performance and resource contention issues that sharing brings.

Keep the detailed pieces in mind (or on a different piece of paper), particularly where the details may have a system impact, but try to keep the details out of the diagram if possible. The goal is to break the system down from a complete thing into bite-sized chunks you can summarize enough to fit in a box.

The next step is to add some higher-level functionality. What is each external chip used for? This is simple when each has only one function. For example, if our flash

is used to store bitmaps to put on a screen, we can put a box on the architecture to represent the display assets. This doesn't have an off-chip element, so its box goes in the processor. We'll also need boxes for a screen and its communication method, and another box to convey flash-based display assets to the screen. It is better to have too many boxes at this stage than too few. We can combine them later

Add any other software structures you can think of: databases, buffering systems, command handlers, algorithms, state machines, etc.

Try to represent everything from the hardware to the product function in the diagram

- **Organigram:** The next type of software architecture diagram, organigram, looks like an organizational chart. In fact, I want you to think about it as an org chart. Like a manager, the upper-level components tell the lower ones what to do. Communication is not (should not be!) only one direction. The lower-level pieces will act on the orders to the best of their ability, provide requested information, and notify their boss when errors arise. Think of the system as a hierarchy, and this diagram shows the control and dependencies.

shows discrete components and which ones call others. The whole system is a hierarchy, with main at the highest level. If you have the device's algorithm planned out and know how the algorithm is going to use each piece, you can fill in the next level with algorithm-related objects. If you don't think they are going to change, you can start with the product-related features and then drop down to the pieces we do know, putting the most complex on top. Next, fill in the lower-level objects that are used by a higher-level object. For instance, our SPI object is used by our flash object, which is used by our display assets object, and so on. You may need to add some pieces here, ones you hadn't thought of before. You should determine whether those pieces need to go on the block diagram, too (probably).

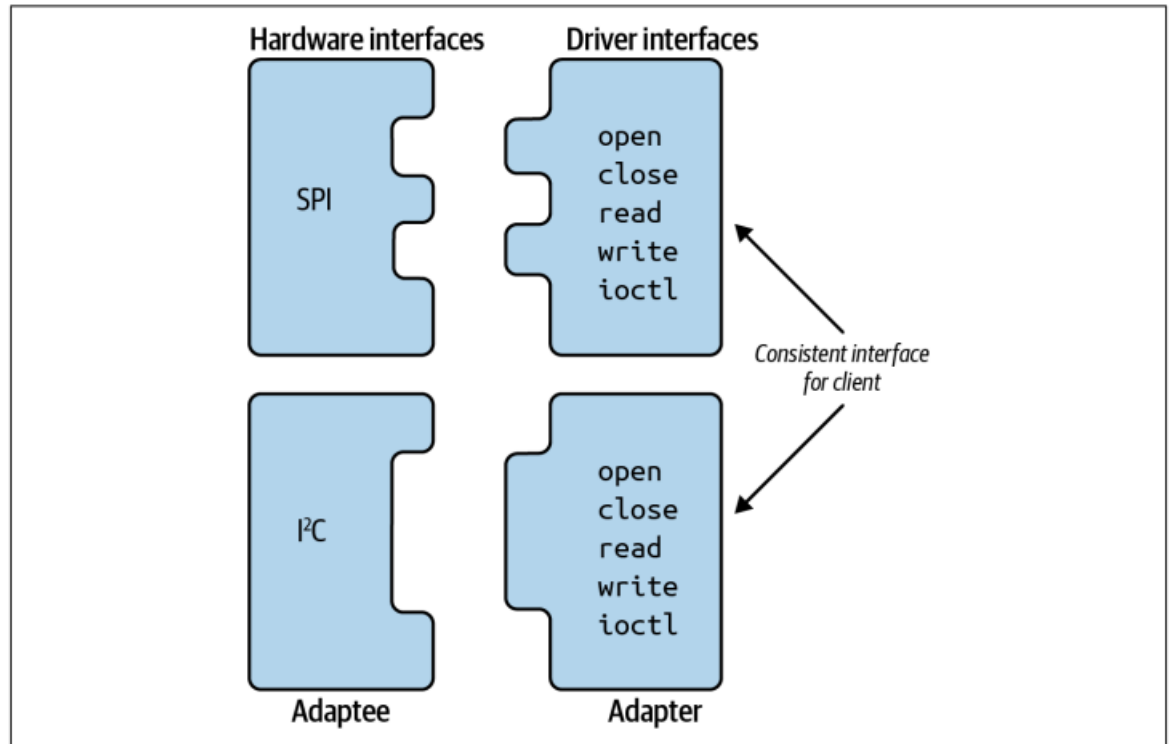


Figure 2-7. Drivers implement the adapter pattern

- **Layering Diagram:** The last architecture drawing looks for layers and represents objects by their estimated size, as in Figure 2-6. This is another diagram to start with paper and pencil. Start at the bottom of the page and draw boxes for the things that go off the processor (such as our communication boxes). If you anticipate that one is going to be more complicated to implement, make it a little larger. If you aren't sure, make them all the same size.

Next, add to the diagram the items that use the lowest layer. If there are multiple users of a lower-level object, they should all touch the lower-level object (this might mean making the lower-level component bigger). Also, each object that uses something below it should touch all of the things it uses, if possible.

- **Driver Interface: Open, Close, Read, Write, IOCTL:** The previous section used a top-down view of module interfaces to train you to consider encapsulation and think about where you can get help from another person. Going from the bottom up works as well. The bottom here consists of the modules that talk to the hardware (the drivers).

Top-down design is when you think about what you want and then dig into what you need to accomplish your goals. Bottom-up design is when you consider what you have and build what you can out of that.

Many drivers in embedded systems are based on the POSIX API used to call devices in Unix systems. Why? Because the model works well in many situations and saves you from reinventing the wheel every time you need access to the hardware.

The interface to Unix drivers is straightforward:

- open Opens the driver for use. Similar to (and sometimes replaced by) init.
- close Cleans up the driver after use.
- read Reads data from the device.
- write Sends data to the device.
- ioctl (pronounced “eye-octal” or “eye-oh-control”) Stands for input/output (I/O) control and handles the features not covered by the other parts of the interface. Its use is somewhat discouraged by kernel programmers due to its lack of structure, but it is still very popular.

In Unix, a driver is part of the kernel. Each of these functions takes an argument with a file descriptor that represents the driver in question (such as /dev/tty01 for the first terminal on the system). That gets pretty cumbersome for an embedded system without an operating system. The idea is to model your driver upon Unix drivers. A sample functionality for an embedded system device might look like any of these:

- spi.open()
- spi_open()
- SpiOpen(WITH_LOCK)
- spi.ioctl_changeFrequency(THIRTY_MHz)
- SpiIoctl(kChangeFrequency, THIRTY_MHz)

This interface straightens out the kinks that can happen at the driver level, making them less specific to the application and creating reusable code. Further, when others come up to your code and the driver looks like it has these functions, they will know what to expect.

The driver model in Unix sometimes includes two newer functions. The first, select (or poll), waits for the device to change state. That used to be done by getting empty reads or polling ioctl messages, but now it has its own function. The second, mmap, controls the memory map the driver shares with the code that calls it.

- **Adapter Pattern:** One traditional software design pattern is called adapter (or sometimes wrapper). It converts the interface of an object into one that is easier for a client to use (a higherlevel module). Often, adapters are written over software APIs to hide ugly interfaces or libraries that change.

Many hardware interfaces are like ungainly software interfaces. That makes each driver an adapter.

If you create a common interface to your driver (even if it isn’t open, close, read, write, ioctl), the hardware interface can change without your upper-level software changing. Ideally, you can switch platforms altogether and need only to rework the underpinnings.

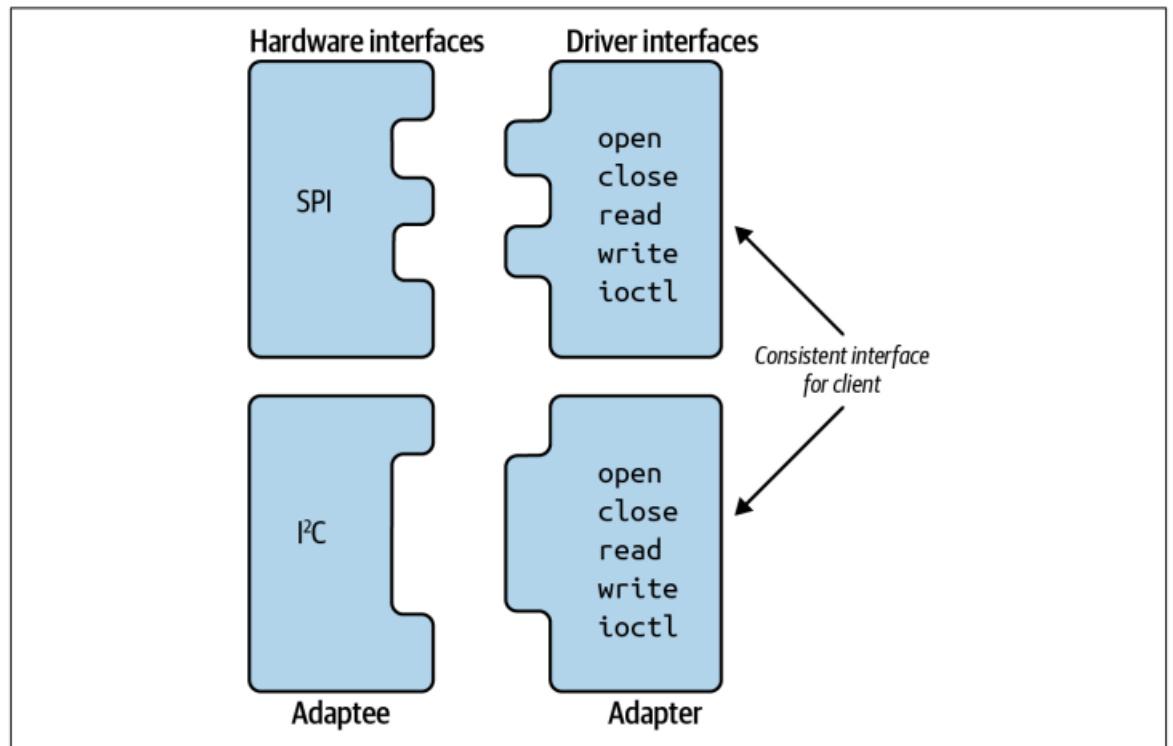


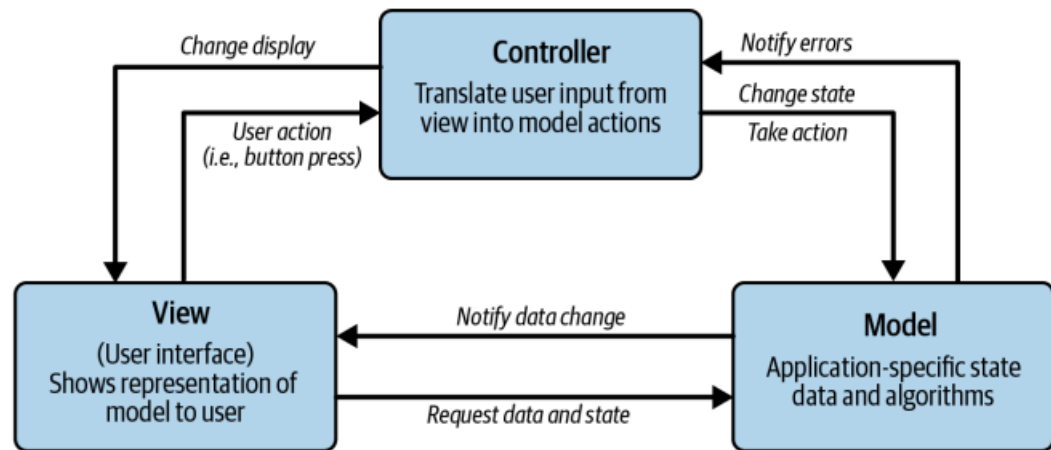
Figure 2-7. Drivers implement the adapter pattern

Note that drivers are stackable. In our example we have a display that uses flash memory, which in turn uses SPI communication. When you call open for the display, it will call its subsystems initialization code, which will call open for the flash, which will call open for the SPI driver. That's three levels of adapters, all for the cause of portability and maintainability.

The layers and the adapters add complexity to the implementation. They may also require more memory or add delays to your code. That is a trade-off. You can have simpler architectures with all of the good maintainability, testing, portability, and so on for a small price. Unless your system is very resource constrained, this is usually a good trade.

- **Singleton pattern:** Singleton pattern is a software design pattern that ensures a class has only one instance and provides a global access point to that single instance.
 - **Private Constructor:** Blocks external code from using new to make a new instance.
 - **Private Static Field:** Holds the single reference to the class object in memory.
 - **Public Static Creation Method:** Acts as a global gateway that checks if the instance exists, creates it if it is missing, and returns the same object every time.
- **MVC (Model view controller) pattern:** In embedded systems, the Model-View-Controller (MVC) pattern is an architectural design choice used to separate an application's core hardware logic, user display interface, and input handling.
 - **Model (The System State & Data Logic):** Represents the core application logic, data structures, and the state of the physical hardware.

- **View (The Output Interface):** Responsible for presenting the system state to the user.
- **Controller:** Acts as the middleman or liaison. It receives user input, talks to the model to get or change data, and decides which view to show next.



Getting your hands on the hardware

- **Intro:** It can be tough to start working with embedded systems. Most software engineers need a crash course in electrical engineering, and most electrical engineers need a crash course in software design. Working closely with a member of the opposite discipline will make getting into embedded systems much easier.
- **Hardware Design:** Once they know what they are building, the hardware team goes through datasheets and reference designs to choose components, ideally consulting the embedded software team. Often development kits are purchased for the riskiest parts of the system, usually the processor and the least-understood peripherals

The hardware team creates schematics, while the software team works on the development boards. It may seem like the hardware team is taking weeks (or months) to make a drawing, but most of that time is spent wading through datasheets to find a component that does what it needs to for the product, at the right price, in the correct physical dimensions, with a good temperature range, and so on. During that time, the embedded software team is finding (or building) a toolchain with compiler and debugger, creating a debugging subsystem, trying out a few peripherals, and possibly building a sandbox for testing algorithms.

Schematics are entered using a schematic capture program (aka CAD package). These programs are often expensive to license and cumbersome to use, so the hardware engineer will usually generate a PDF schematic at checkpoints or reviews for your use. Although you should take the time to review the whole schematic, the part that has the largest impact on you is the processor and its view of the system. Because hardware engineers understand that, most will generate an I/O map that describes the connection attached to each pin of the processor.

The schematic shows the electrical connections but it doesn't specify exactly which parts need to be on the board. It may show a header with ten pins, but for the boards to be assembled, the manufacturer will need to know exactly what kind of header, usually with a link to a datasheet specifying everything about it. All of the components are on the bill of materials (BOM), something that is started during schematic capture.

Once the schematic is complete (and the development kits have shown that the processor and risky peripherals are probably suitable), the board can be laid out. In layout, the connections on the schematic are made into a drawing of physical traces on a board, connecting each component as it is in the schematic.

Board layout is often a specialized skill, different from electrical engineering, so don't be surprised if someone other than your hardware engineer does the board layout.

After layout, the board goes to fabrication (fab), where the printed circuit boards (PCBs) are built. Then they get assembled with all of their components. Gathering the items on the BOM so you can start assembly is called putting together the kits. Getting the parts with long lead times can make it difficult to create a complete kit, which delays assembly. An assembled board is called a PCBA (printed circuit board assembly), or just a PCA, which is what will sit on your desk.

When the schematic is done and the layout has started, the primary task for the embedded software team is to define the hardware tests and get them written while the boards are being created. Working on the user-facing code may seem more productive, but when you get the boards back from fabrication, you won't be able to make progress on the software until you bring them up. The hardware tests will not only make the bring-up smoother, but they also will make the system more stable for development (and production). As you work on the hardware tests, ask your electrical engineer which parts are the riskiest. Prioritize development for the tests for those subsystems (and try them out on a development kit, if you can).

When the boards come back, the hardware engineer will power them on to verify there aren't power issues, possibly verifying other purely hardware subsystems. Then you get a board (finally!) and bring-up starts.

- **Board Bring-Up:** Board bring-up is the initial process of powering on, testing, and validating a newly manufactured or revised printed circuit board (PCB) for the very first time to ensure the hardware and basic software work together

This can be a fun time, one where you are learning from an engineer whose skill set is different from yours. Or it can be a shout-fest where each team accuses the other of incompetence and maliciousness. Your call.

The board you receive may or may not be full of bugs. Electrical engineers don't have an opportunity to compile their code, try it out, make a change, and recompile. Making a board is not a process with lots of iterations. Remember that people make mistakes, and the more fluffheaded the mistake, the more likely it will take an eternity to find and fix

To make bring-up easier on both yourself and your hardware engineer, first make each component individually testable. If you don't have enough code space, make another software subproject for the hardware test code (but share the underlying modules as much as possible).

Second, when you get the PCBA, start on the lowest-level pieces, at the smallest steps possible. For example, don't try out your fancy motor controller software. Instead, set an I/O device to go on for a short time, and hook up an LED to make sure it flashes. Then (still taking things one step at a time), do the minimum necessary to make sure the motor moves (or at least twitches).

- **Reading a Datasheet:** If you are a software engineer, consider each chip as a separate software library. Think of all the effort you would need to put into learning a software package (Qt, OpenCV, Zephyr, Unity, TensorFlow, etc.). That is about how long it will take to become familiar with your processor and its peripherals. The processors will even have methods of talking to the peripherals that are somewhat like APIs. As with libraries, you can often get away with reading only a subset of the documentation—but you are better off knowing which subset is the most important one for you before you end up down a rabbit hole.

Datasheets are the API manuals for peripherals. Make sure you have the latest version of your datasheet (check the manufacturer's site and do an online search just to be sure) before I let you in on the secrets of what to read twice and what to ignore until you need it.

Unfortunately, some manufacturers release their datasheets only under NDA, so their websites contain only an overview or summary of their product.

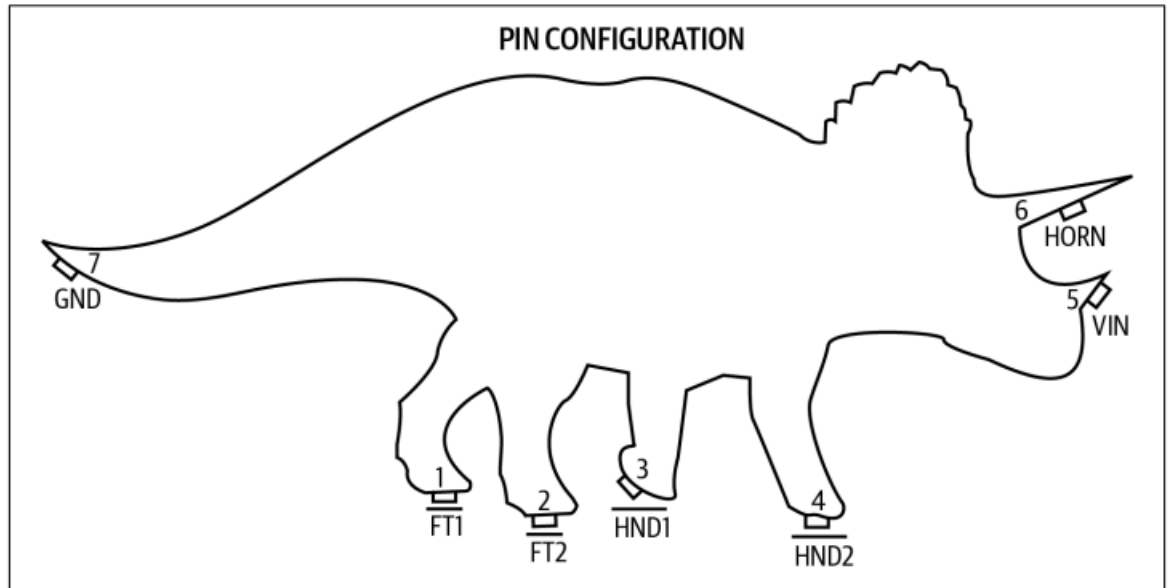
Hardware components are described by their datasheets. Datasheets can be intimidating; they are packed densely with information. Reading datasheets is an art, one that requires experience and patience. The first thing to know about datasheets is that they aren't really written for you. They are written for an electrical engineer, more precisely for an electrical engineer who is already familiar with the component or its close cousin

- **Datasheet Sections You Need When Things Go Wrong:** After the first page, the order of information in each datasheet varies. And not all datasheets have all sections. Nonetheless, you don't want to scrutinize sections that won't offer anything to help your software. Save time and mental fatigue by skipping over the absolute maximum ratings, recommended operating conditions, electrical characteristics, packaging information, layout, and mechanical considerations. The hardware team has already looked at that part of the sheet; trust them to do their jobs.

Although you can skip the following sections now, remember that these sections exist. You may need them when things go wrong, when you have to pull out an oscilloscope and figure out why the driver doesn't work, or (worse) when the part seems to be working but not as advertised. Note that these sections exist, but you can come back to them when you need them; they are safe to ignore on the first pass.

- **Pinout:** A pinout is a map or chart that shows the physical pins on an electronic chip or board and explains what job each individual pin does.
- **Pin probing:** To probe a pin means to touch or insert a specialized metal test pin into an electrical terminal, connector, or circuit point to measure voltage, resistance, or signals without damaging the hardware
- **Pinout for each type of package available:** You'll need to know the pinout if you have to probe the chip during debugging. Ideally, this won't be important, because your software will work and you won't need to probe the chip.

The pin configuration shows some pin names with bars over them



The bars indicate that these pins are active low, meaning that they are on when the voltage is low. In the pin description, active low pins have a slash before the name. You may also see other things to indicate active low signals. If most of your pins have a NAME, look for a modifier that puts the name in the format nNAME, #NAME, _NAME NAME*, NAME_N, etc. Basically, if it has a modifier near the name, look to see whether it is active low, which should be noted in the pin descriptions section.

- **Oscilloscope:** An oscilloscope is an electronic test instrument that graphs electrical voltage changes over time on a two-dimensional display
- **Datasheet Sections for Software Developers:** Eventually, possibly halfway through the datasheet, you will get to some text (Figure 3-5). This could be titled “Application Information” or “Theory of Operation.” Or possibly the datasheet just switches from tables and graphs to text and diagrams. This is the part you need to start reading. It is fine to start by skimming this to see where the information you need is located, but eventually you will really need to read it from start to end. As a bonus, the text there may link to application notes and user manuals of value. Read through the datasheet, considering how you’ll need to implement the driver for your system. How does it communicate? How does it need to be initialized? What does your software need to do to use the part effectively? Are there strict timing requirements, and how can your processor handle them?

As you read through the datasheet, mark areas that may make an impact on your code so you can return to them. Timing diagrams are good places to stop and catch your breath

In addition to making sure you have the latest datasheet for your component, check the manufacturer’s web page for errata, which provide corrections to any errors in the datasheet

Search the web page for the part number and the word “errata” (Figure 3-7). Errata can refer to errors in the datasheet or errors in the part. Most parts have some sort of errata.

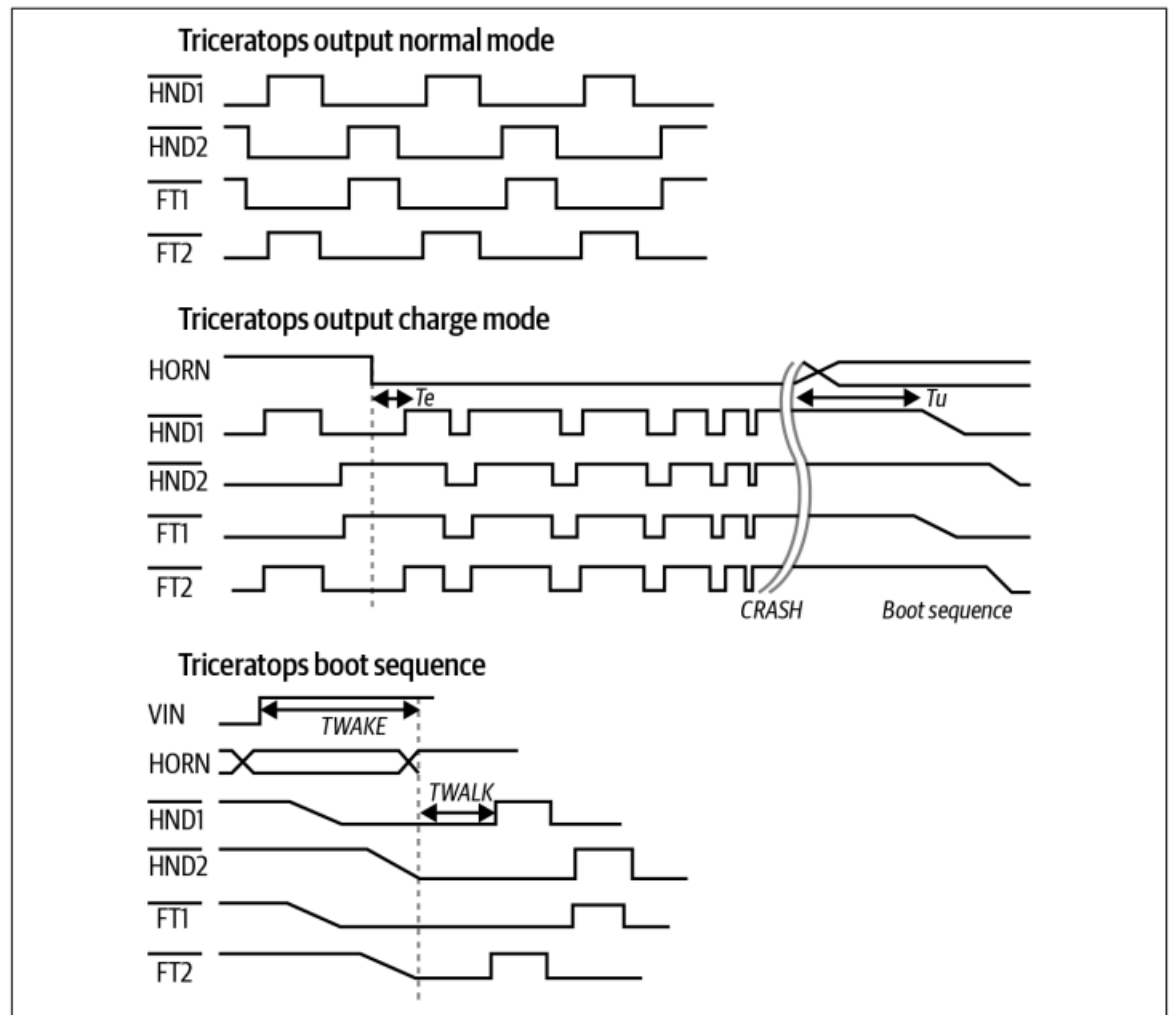


Figure 3-6. Analog Triceratops timing diagrams

When you are done, if you are very good or very lucky, you will have the information you need to use the chip. Generally, most people will start writing the driver for the chip and spend time re-reading parts as they write the interface to the chip, then the communication method, then finally actually use the chip. Reading datasheets is a race where the tortoise definitely wins over the hare.

Once you are all done with the driver and it is working, read through the feature summary at the top of the datasheet because now you have conquered this type of component and can better understand the summary. Even so, when you implement something similar, you'll probably still need to read parts of the datasheet, but it will be simpler and you'll get a much better overview from the summary on the datasheet's front page.

Analog Triceratops Datasheet Errata

Classifications/corrections to the datasheet

In the device datasheet listed below, the following clarifications and corrections should be noted. The component Triceratops may mature over time into the Torosaurus as light and temperature cycles cause the frills to develop fenestrae (holes). To work around the issue, the frill is not recommended for use in the Triceratops.

Device	Datasheet
<i>T. horridus</i> <i>T. prorsus</i>	Analog Triceratops
<i>T. latus</i> <i>T. utahensis</i>	Analog Torosaurus

Figure 3-7. Analog Triceratops errata

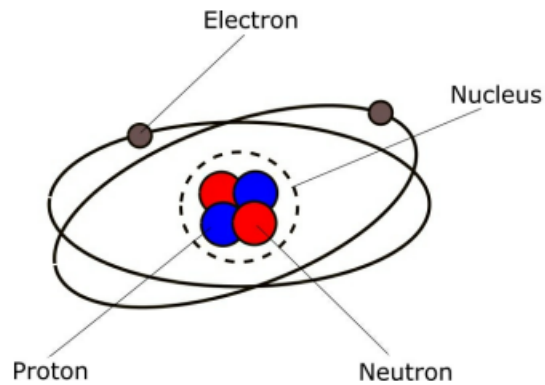
Other resources may also be available as you work with peripherals:

- If the chip has a user manual, be sure to look at that.
- Application notes often have specific use cases that may be of interest. These often have vendor example code associated with them. Also look for a vendor code repository.
- Search the part on GitHub or the web. Example code from other folks using the part can show you what you should be doing.
- **Your Processor Is a Language:** The most important component is the one your software runs on: your processor.

Some vendors (ST, TI, Microchip, NXP, and so on) use a common core (i.e., Arm Cortex-M4F). The vendors provide different on-chip peripherals. How your code interfaces with timers or communication methods will change. Sharing a core will make the change less difficult than moving between a tiny 8-bit processor and a mighty 64-bit processor, but you should expect differences

- **Reading a Schematic:** If you come from the traditional software world, schematics can seem like an eye chart with hieroglyphics, interspersed with strange boxes and tangled lines. As with a datasheet, knowing where to start can be daunting. Beginning on page one of a multipage schematic can be dicey because many electrical engineers put their powerhandling hardware there, something you don't necessarily care about. Figure 3-9 shows you a snippet of one page of a schematic.

Some schematics have blocks of textual data (often in the corner of a page). Those are comments; read them carefully, particularly if the block is titled “Processor I/Os” or something equally useful. These are like block comments in code: they aren't always there, so you should take your electrical engineer out to lunch when their schematic is well commented.



Atoms are really small, ranging in size from about 0.1 to 0.5 nanometers (1×10^{-10} m to 5×10^{-10} m)

That's roughly a million times thinner than a human hair

Protons and neutrons are even smaller, with electrons being the smallest particles

An electron is about 200,000 times smaller than a proton

Figure 1 - The structure of an atom

As you go through a schematic for the first time, start by looking for boxes with lots of connections. Often this can be simplified further because box size on the schematic is proportional to the number of wires, so look for the largest boxes. Your processor is likely to be one of those things. Because the processor is the center of the software world, finding it will help you find the peripherals that you most care about.

Inputs, outputs, and timers

- **Intro:** Inputs, outputs, and timers form the basis for almost everything embedded systems do. Even the communication methods to talk to other components are made up of these
- **Handling Registers:** To do anything with an I/O line, we need to talk to the appropriate register.

Described in the chip's user manual, registers come in all flavors to configure the processor and control peripherals. They are memory-mapped, so you can write to a specific address to modify a particular register. Often each bit in the register means something specific, which means we need to start using binary numbers to look at the values of specific bits.

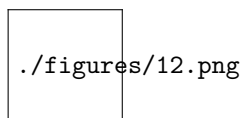
- **Test, Set, Clear, and Toggle:** Hex and bitwise operations are only a lead-up to being able to interact with registers. If you want to know if a bit is set, you need to bitwise AND with the register:

If you want to set a bit in a register, OR it with the register:

Clearing a bit is quite a bit more confusing because you want to leave the other bits in the register unchanged. The typical strategy is to invert the bit we want to clear using bitwise NOT ($\sim$). Then AND the inverted value with the register. This way, the other bits remain unchanged:

Toggle a bit using the XOR operation.

- **Toggling an Output:** Most processors have pins whose digital states can be read (input) or set (output) by the software. These go by the name of I/O pins, general-purpose I/O (GPIO), digital I/O (DIO), and, occasionally, general I/O (GIO). The basic use case is usually straightforward, at least when an LED is attached:
 1. Initialize the pin to be an output (as an I/O pin, it could be input or output).
 2. Set the pin high when you want the LED on. Set the pin low when you want the LED off. (Although the LED can also be connected to be on when the pin is low, this example will avoid inverted logic.)
- **Setting the Pin to Be an Output:** most I/O pins can be either inputs or outputs. The first register you'll need to set will control the direction of the pin so it is an output. First, determine which pin you will be changing. To modify the pin, you'll need to know the pin name (e.g., "I/O pin 2," not its number on the processor (e.g., pin 12). The names are often inside the processor in schematics, while the pin number is on the outside of the box



The pins may have multiple numbers in their name, indicating a port (or bank) and a pin in that port. (The ports may also be letters instead of numbers.) In the figure, the LED is attached to processor pin 10, which is named "SCLK/IO1_2." This pin is shared between the SPI port

and the I/O subsystem (IO1_2). The user manual will tell you whether the pin is an I/O by default or a SPI pin (and how to switch between them). Another register may be needed to indicate the purpose of the pin. Most vendors are good about cross-referencing the pin setup, but if it is shared between peripherals, you may need to look in the peripheral section to turn off unwanted functionality. In our example, we'll say the pin is an I/O by default.

In the I/O subsystem, it is the second pin (2) of the first bank (1). We'll need to remember that and to make sure the pin is used as an I/O pin and not as a SPI pin. Your processor user manual will describe more. Look for a section with a name like "I/O Configuration," "Digital I/O Introduction," or "I/O Ports." If you have trouble finding the name, look for the word "direction," which tends to be used to describe whether you want the pin to be an input or an output.

Once you find the register in the manual, you can determine whether you need to set or clear a bit in the direction register. In most cases, you need to set the bit to make the pin an output. You could determine the address and hardcode the result:

```
0  *((int*)0x0070C1) |= (1 << 2);
```

However, please don't do that. The processor or compiler vendor will almost always provide a header file or a hardware abstraction layer (HAL, which includes a header file) that hides the memory map of the chip so you can treat registers as global variables. If the vendor didn't give you a header file, make one for yourself, so that your code looks more like one of these lines:

```
0  // STM32F103 processor
1  GPIOA->CRL |= 1 << 2; // set IOA_2 to be an output
2
3  // MSP430 processor
4  P1DIR |= BIT2; // set IO1_2 to be an output
5
6  // ATtiny processor
7  DDRB |= 0x4; // set IOB_2 to be an output
```

Note that the register names are different for each processor, but the effect of the code in each line is the same. Each processor has different options for setting the second bit in the byte (or word).

In each of these examples, the code is reading the current register value, modifying it, and then writing the result back to the register. This read-modify-write cycle needs to happen in atomic chunks, meaning they need to execute without interruptions or any

other processing between steps. If you read the value, modify it, and then do some other stuff before writing the register, you run the risk that the register has changed and the value you are writing is out of date. The register modification will change the intended bit, but also might have unintended consequences.

- **Turning on the LED:** The next step is to turn on the LED. Again, we'll need to find the appropriate register in the user manual:

```

0 // STM32F103 processor
1 GPIOA->ODR |= (1 << 2); // IOA_2 high
2
3 // MSP430 processor
4 P1OUT |= BIT2; // IO1_2 high
5
6 // ATtiny processor
7 PORTB |= 0x4; // IOB_2 high

```

The GPIO hardware abstraction layer header file provided by the processor vendor shows how the raw addresses get masked by some programming niceties. In STM32F103x6, the I/O registers are accessed at an address through a structure (I've reorganized and simplified the file):

```

0 typedef struct
1 {
2     __IO uint32_t CRL; // Port configuration (low)
3     __IO uint32_t CRH; // Port configuration (high)
4     __IO uint32_t IDR; // Input data register
5     __IO uint32_t ODR; // Output data register
6     __IO uint32_t BSRR; // Bit set/reset register
7     __IO uint32_t BRR; // Bit reset register
8     __IO uint32_t LCKR; // Port configuration lock
9 } GPIO_TypeDef;
10
11 #define PERIPH_BASE 0x40000000UL
12 #define APB2PERIPH_BASE (PERIPH_BASE + 0x00010000UL)
13 #define GPIOA_BASE (APB2PERIPH_BASE + 0x00000800UL)
14 #define GPIOA ((GPIO_TypeDef *)GPIOA_BASE)

```

The header file describes many registers we haven't looked at. These are also in the user manual section, with a lot more explanation. I prefer accessing the registers via the structure because it groups related functions together, often letting you work with the ports interchangeably

Once we have the LED on, we'll need to turn it off again. You just need to clear those same bits

```

0 // STM32F103 processor
1 GPIOA->ODR &= ~(1 << 2); // IO1_2 low

```

- **Blinking the LED:** To finish our program, all we need to do is put it all together. The pseudocode for this is as follows:


```
0  main:
1      initialize the direction of the I/O pin to be an output
2  loop:
3      set the LED on
4      do nothing for some period of time
5      set the LED off
6      do nothing for the same period of time
7      repeat loop
```

Once you've programmed it for your processor, you should compile, load, and test it. You might want to tweak the delay loop so the LED looks about right. It will probably require several tens of thousands of processor cycles, or the LED will blink faster than you can perceive it.